Laborator 04 - Git și Github

Version Control

Un sistem pentru controlul versiuni este un sistem care înregistrează modificările suferite de un fișier sau un grup de fișiere în decursul timpului pentru a facilita revenirea la o versiune specifică ulterior.

Există 3 tipuri de sisteme pentru controlul versiunii (VCS – Version Control System):

Sisteme locale pentru controlul versiunii

Cele mai vechi VCS-uri au presupus că toți dezvoltatorii lucrau în propriile directoare pe un singur sistem de fișiere partajat. Repository-ul a fost păstrat într-un director separat (adesea ascuns). A face check out şi a trimite commit-uri se rezolvă prin operații obișnuite cu fișiere, precum copieri și redenumiri.

Sisteme centralizate pentru controlul versiunii

Sistemele locale pentru controlul versiunii au oferit o soluție pentru gestiunea versiunii pentru aplicații dezvoltate de o singură persoană pe un anume sistem. Din păcate această soluție nu vine în ajutorul persoanelor ce trebuie să colaboreze cu alți dezvoltatori și să interacționeze cu alte sisteme. Pentru a rezolva și această problemă sistemele centralizate pentru controlul versiunii (CVCS – Centralized Version Control Systems) au fost dezvoltate.

Sistemele centralizate pentru controlul versiunii (CVCS – Centralized Version Control Systems) (precum CVS, Subversion, Perforce etc) au repository-ul pe un singur server, conectat la retea. Pentru a face check out, fisierele trebuie transferate din server în directorul de lucru.

Aceasta soluție oferă numeroase avantaje comparativ cu sistemele locale pentru controlul versiunii: toată lumea știe până la un anumit nivel ce fac restul colaboratorilor în cadrul proiectului, administratorii au la dispoziție o manieră eficientă de a gestiona accesul la resurse și este mult mai ușor de gestionat o bază de date centrală decât una locală pentru fiecare utilizator.

O problemă evidentă este legată de disponibilitatea nodului central: dacă acesta întâmpină orice fel de problemă o să împiedice orice acțiune din partea utilizatorilor până problema va fi rezolvată. În plus fiind singurul sistem care conține toate stările fișierelor din cadrul unui proiect, în cazul în care aceste date sunt pierdute recuperarea acestora este de foarte multe ori imposibilă.

Sisteme distribuite pentru controlul versiunii

Dezavantajele și riscurile create de un sistem centralizat pentru controlul versiunii a condus la apariția sistemelor distribuite pentru controlul versiunii (Distributed Version Control Systems – DVCS)(precum Git, Mercurial, Bazaar, Darcs etc), în cadrul acestora utilizatorii nu obțin doar ultima versiune a fișierelor, ci propria copie a repository-ului cu toate versiunile noi și vechi ale proiectului. Diferitele repository-uri se sincronizează între ele periodic pe rețea.

Un avantaj este că această abordare adaugă robustețe. În modelele locale și centralizate, un repository corupt sau un hard disk eșuat poate duce la pierderea tuturor datelor. În modelul distribuit, vă puteți recupera adesea din astfel de accidentări, deoarece mai mulți dezvoltatori își pot uni repository-le pentru a recupera întregul istoric al proiectului. Majoritatea echipelor care lucrează cu VCS distribuite vor desemna în continuare un repository ca origine centrală a copiilor lor, pur și simplu pentru că acest lucru simplifică comunicarea în echipă.

Avantajele sistemelor de versionare

Care sunt avantajele folosirii unui sistem de versionare? Este salvat istoricul tuturor modificărilor, astfel că se poate reveni oricând la o versiune mai veche dacă se descoperă prin folosirea unui serviciu de hosting, codul sursă are mereu o copie de siguranță online cea mai recentă versiune a codului sursă este mereu disponibilă tuturor dezvoltatorilor, făcând astfel colaborarea și sincronizarea mult mai ușoară decât în cazul trimiterii de fișiere conținând cod sursă dezvoltatorilor interesați de proiect.

GitHub

GitHub este o platformă online, pe care dezvoltatorii o pot folosi pentru a stoca și versiona codul lor sursă. Git este un sistem de management și versionare a codului sursă care permite lucrul eficient la un proiect software. Astfel, Git este utilitarul folosit în terminal, iar GitHub este serverul și aplicația web pe care rulează acesta, locul în care păstrăm repository-ul remote.

Pregătirea inițială a mediului Git

```
$ git config --global user.name "Prenume Nume"
$ git config --global user.email "adresa_de_email@example.com"
$ git config --list
```

Crearea unui fork al unui repository

Noţiunea de **fork** nu este specifică git-ului, ci platformelor care stochează repository-uri, precum Github si Gitlab. Fork reprezintă o copie a unui repository, unde creatorul fork-ului poate face modificări, chiar daca nu are drepturi de a modifică repository-ul original. Pentru a crea un fork al unui repository stocat pe GitHub, navigaţi pe pagina repository-ului, apoi apăsaţi butonul “Fork” din dreapta sus.

Clonarea unui repository

```
$ git clone <url_repository>
```

Remote, origin and upstream

Termenul de **remote** se referă la locaţia unde este stocat un repository, pe un server de git. La clonarea unui repository de pe GitHub, este adăugat în mod automat un remote, numit **origin**. În cazul unui repository obţinut dintr-un fork, **origin** va fi reprezentat de locaţia fork-ului. Orice modificare la repository-ul original nu se va vedea automat şi în copie. Pentru a putea sincroniza cele 2 repository-uri, se foloseşte un alt **remote**, numit de obicei **upstream**.

Pentru a adăuga un **remote** numit **upstream**, se poate folosi comanda urmatoare, in folder-ul unui repository:

```
$ git remote add upstream <url_repository_original>
```

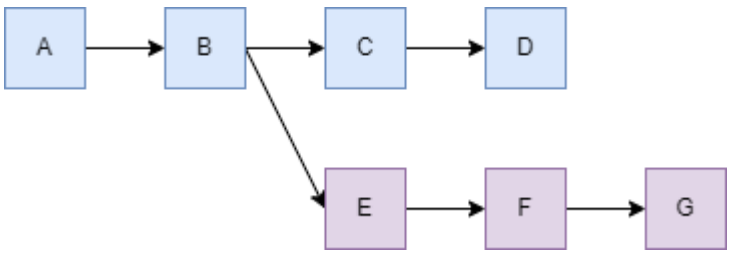
Sincronizare

Pentru a aduce modificarile dintr-un repository remote intr-unul local, se foloseste comanda.

```
$ git pull
```

În cazul în care pe repository-ul local este activ un **branch**, comanda de mai sus va sincroniza branch-ul local cu cel remote.

Există situaţii în care, fiind pe un branch copil, pornit dintr-un branch părinte (ex. **main**), dorim să integram modificările din părinte în branch-ul nostru. Aici intervine noţiunea de **rebase**.



Rebase

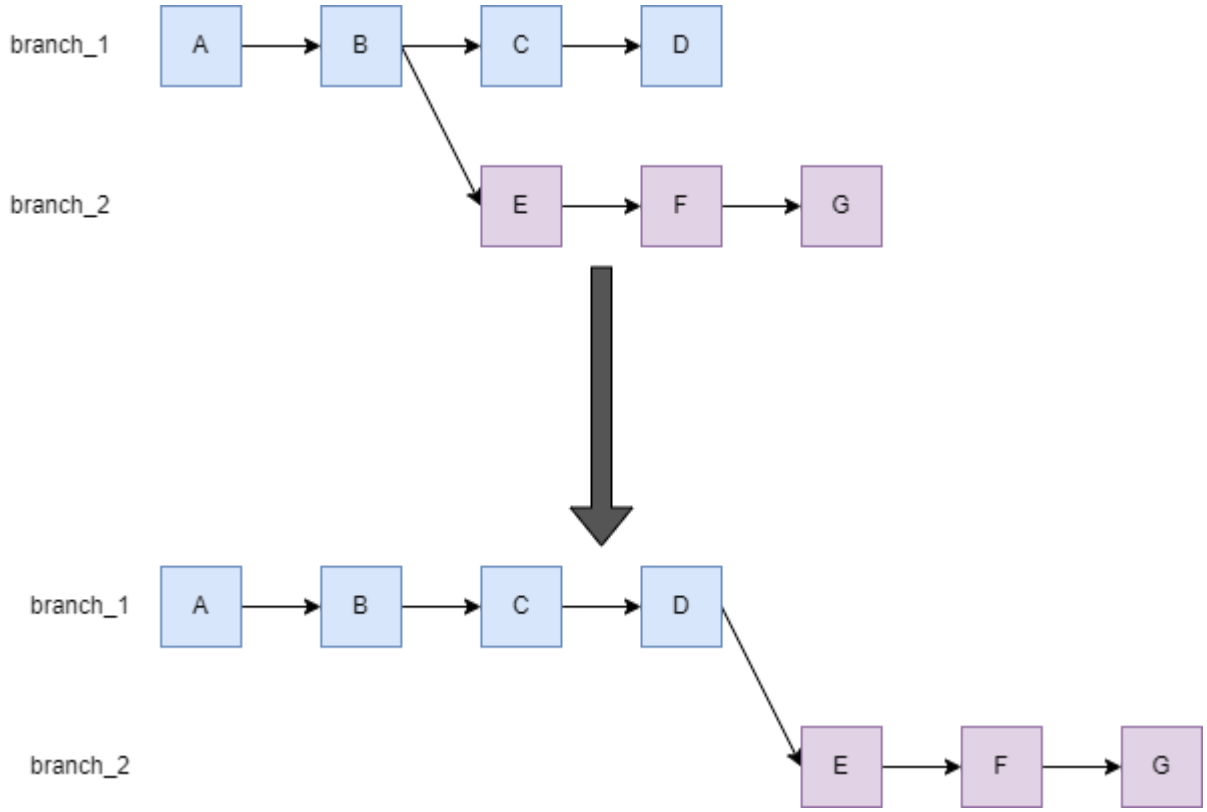
Prin **rebase** se înţelege schimbarea bazei unui branch. Baza unui branch este ultimul commit comun dintre branch-ul copil şi branch-ul părinte.

Cea mai simpla metoda de a integra schimbarile dintr-un branch parinte intr-unul copil este prin folosirea comenzii

```
$ git pull --rebase <remote> <nume_parinte>
```

Pentru a aduce schimbările din branch-ul **main** din **remote**-ul **origin**, comanda devine

```
$ git pull --rebase origin main
```



După rebase, dacă dorim să trimitem modificările către remote, folosind comanda

```
$ git push
```

vom observa că istoricul de commit-uri nu este cum ne-am aştepta.

Pentru a avea un istoric curat, e nevoie se folosim comanda

```
$ git push --force
```

Aceasta variantă de **push** va forţa suprascrierea istoricului de pe remote cu cel de pe local.

Stashing

Varianta de mai sus funcţionează doar daca schimbările locale sunt sub forma unui commit. În cazul în care există fişiere modificate, dar care nu au fost adăugate într-un commit, comanda de **pull** va eşua. Pentru asta există 2 soluţii:

1. Crearea unui commit - nu este întodeauna de dorit (nu este finalizat ce vrem sa adăugăm)
2. Folosirea funcţionalităţii de **stash**

Stash ne permite să salvăm modificările care nu fac parte dintr-un commit.

Pentru a salva toate modificările dintr-ul repository, se foloseşte comanda

```
$ git stash push *
```

Pentru a salva doar un fişier, se foloseşte comanda

```
$ git stash push <fişier>
```

Restaurarea modificărilor salvate se face cu comanda

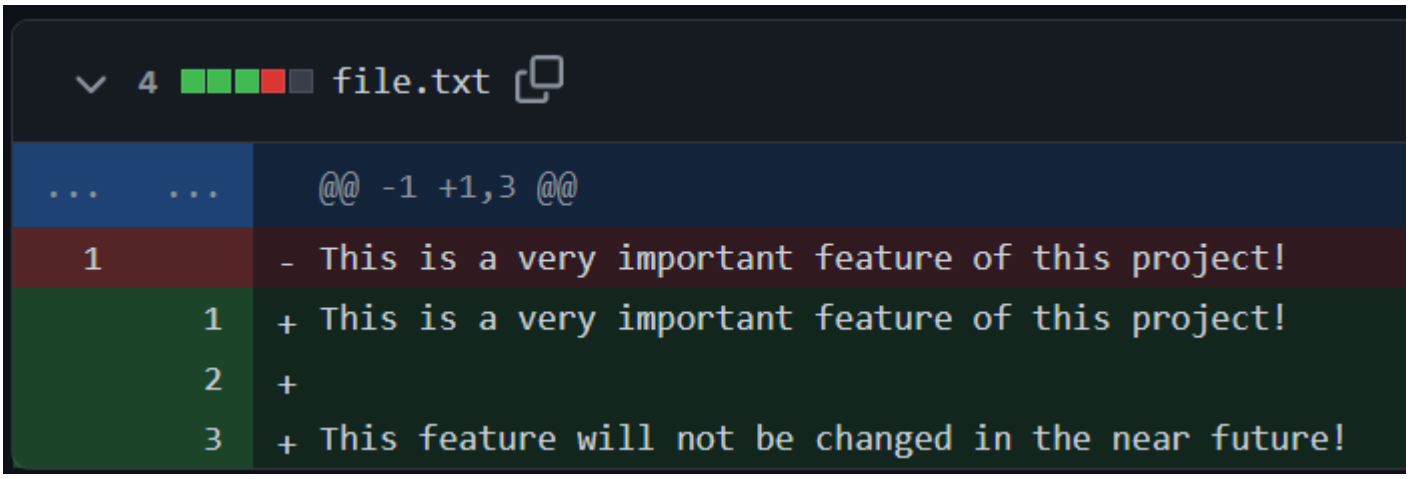
```
$ git stash pop
```

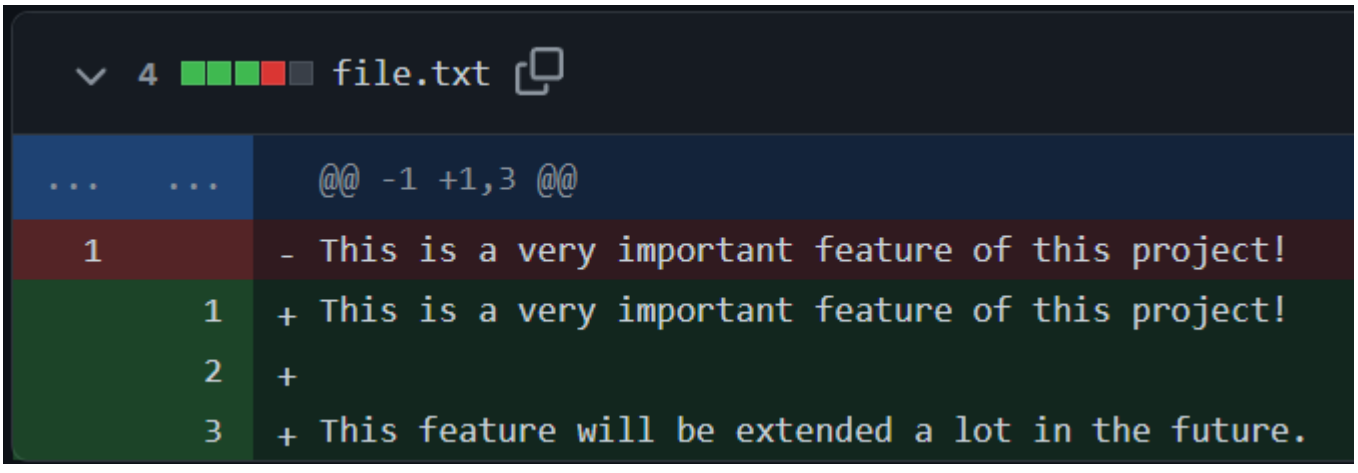
Deoarece acest mecanism funcţioneaza ca o stivă, comanda de **pop** va restaura doar fişierele salvate de ultima comanda de **push**.

Merge conflict

De multe ori, o operaţie de **rebase** sau de **stash pop** va duce la un **merge conflict**. Acesta apare deoarece ultimul commit, **HEAD**, şi modificările pe care vrem să le aplicăm diferă, iar diferenţele nu apar datorită modificărilor noastre (nu fac parte dintr-un **patch**). Pentru a inţelege aceasta situatie, vom urmări un exemplu.

Repository-ul curent conţine 2 branch-uri, **feature_branch_1** şi **feature_branch_2**. Iniţial, pe **feature_branch_1** a fost adăugat fişierul **file.txt**. Branch-ul **feature_branch_2** a fost creat după adăugarea fişierului. Apoi, pe ambele branch-uri au fost făcute modificări asupra fişierului.





Dacă se decide integrarea schimbărilor de pe branch-ul 1 pe branch-ul 2, va apărea un merge conflict. Acesta va fi indicat de git, în fişierele în care se află conflictele.

```
cristi@DESKTOP-8SBBJOK:~/git-lab$ cat file.txt
This is a very important feature of this project!

<<<<<<< HEAD
This feature will not be changed in the near future!
=======
This feature will be extended a lot in the future.
>>>>>>> 96c4da2 (Add second branch modification)
```

Pentru a rezolva un conflict, trebuie urmaţi următorii paşi:

1. Trebuie păstrate una dintre variante, sau ambele, şi eliminate delimitatoarele `<<<<<<< HEAD`, `=======` şi `>>>>>>> <sha>`, apoi salvat fişierul
2. `$ git add <fişier>`
3. `$ git rebase --continue`, dacă merge conflict-ul a apărut în urma unui rebase, sau `git merge --conţinue`, în cazul unui merge.

Organizarea istoricului de commit-uri

Când se adaugă un feature nou unui proiect, este bine ca istoricul de commit-uri să explice schimbările aduse. În practică, mulţi utilizatori realizează commit-uri de tipul “Am rezolvat un bug”, “Am mai rezolvat un bug”. Commit-urile sub forma asta sunt bune pe procesul de dezvoltare, dar nu şi în momentul când feature-ul este integrat în proiectul mare. Se preferă urmatoarea formă a commit-urilor: `[componenta majora]: Funcţionalitate adăugată`. Orice rezolvare de bug, coding-style sau alte modificări, care fac parte din noua funcţionalitate, trebuie integrate într-un singur commit. Asta nu înseamnă că trebuie făcut commit doar când totul merge. Git pune la dispozitie unelte pentru a reface istoricul de commit-uri.

Amend

Comanda

```
$ git commit --amend
```

modifică ultimul commit, prin modificarea mesajului commit-ului, sau a schimbărilor din el. Problema cu această comandă este că în istoric vor apărea atât commit-ul vechi, cât şi cel modificat cu **--amend**. De aceea, e necesară folosirea comenzii `$ git push --force`.

Rebase interactiv

Comanda

```
$ git rebase -i
```

ne permite să modificăm istoricul mai mult decât **--amend**: ne permite să facem amend, să îmbinăm commit-uri (**squash**), să ştergem commit-uri sau chiar să le modificăm ordinea. Comanda mai aşteaptă şi intervalul de commit-uri asupra cărora se vor face modificările. De obicei, acest interval este dat sub forma **@~N**, care înseamnă “ultimele N commit-uri”.

Pentru a face rebase interactiv asupra ultimelor 10 commit-uri, se foloseste comanda

```
$ git rebase -i @~10
```

In urma comenzii, se va deschide un fisier, ca cel de mai jos.

Pentru a modifica un commit, sau pentru a face squash, trebuie inlocuit cuvantul “pick” cu una din comenzile comentate sub lista de commit-uri. Pentru a face squash, trebuie marcate cu “s” commit-urile pe care vrem sa le eliminam. Ele se vor adauga peste primul commit de deasupra marcat cu “pick”. Pentru a incepe rebase-ul interactiv, salvati si inchideti fisierul.

```
  GNU nano 6.2
pick a953e66 Add a missing newline
pick be424bd Make the makefile more generic
pick 5a57c8e Make the makefile fancier
pick a126dc8 Improve adding 5 to the number
pick 3f63e9d Fix compilation
pick c1e8856 Add a gitignore
pick f4eb580 Improve the algorithm
pick 6811d46 Modularize the code
pick bda81a5 Add more random number generation methods
pick 038001c Fix compilation

# Rebase 7ce8e64..038001c onto 7ce8e64 (10 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                    commit's log message, unless -C is used, in which case
#                    keep only this commit's message; -c is same as -C but
#                    opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .       create a merge commit using the original merge commit's
# .       message (or the oneline, if no original merge commit was
# .       specified); use -c <commit> to reword the commit message
#
```

Reset

Există situaţii când este mai simplu să fie refăcut istoricul de commit-uri ştergandu-le pe toate şi rescriindu-le. Pentru asta se poate folosi

```
$ git reset <interval>
```

Această comandă şterge commit-urile, dar păstrează modificările aduse de acele commit-uri. Există varianta

```
$ git reset --hard <interval>
```

care şterge şi modificările aduse fişierelor.

Bisect

Uneori (de cele mai multe ori) proiectele vor avea bug-uri. De cele mai multe ori, vom scrie mult cod, multe commit-uri, apoi vom testa ce am scris. Problema apare când s-au făcut modificări substanţiale şi e nevoie să fie găsit un bug. În situaţiile în care nu se poate observa citind codul, e nevoie să trecem prin tot istoricul de commit-uri, unul câte unul, să vedem unde a apărut problema. Git pune la dispoziţie o unealtă pentru a simplifica procesul: **git bisect**. Această comandă face o cautare binară într-un interval de commit-uri definit de utilizator, căutând primul commit “rău”.

Pentru a folosi **git bisect**, trebuie urmaţi următorii paşi:

1. `$ git bisect start`
2. `$ git bisect bad`

3. `$ git checkout <sha_ultimul_commit_bun>`
4. `$ git bisect good`

După aceste comenzi, git va trece prin fiecare commit şi va aştepta să fie marcat ca **good** sau **bad**, până va găsi primul commit **bad** din istoric.

Pentru a reveni la starea de dinainte de **git bisect start**, se foloseşte comanda **git bisect reset**.

Exerciţii:

- Faceţi fork repository-ului [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/git-lab], cu opţiunea de a copia şi branch-urile existente. Clonaţi repository-ul obtinut. Adaugati un remote **upstream** catre repository-ul original.
- Echipa a decis ca nu se vor mai adăuga feature-uri în plus. Faceţi schimbarea pe branch-ul **feature_branch_2**, pentru a fi la zi cu modificările de pe branch-ul **feature_branch_1**. Păstraţi linia cu “big boss”. Creaţi un pull-request, din interfaţa GitHub, către branch-ul **feature_branch_1** din fork-ul vostru. Adăugaţi asistentul de la laborator ca reviewer.

Pentru a aduce modificarile de pe branch-ul `feature_branch_1` pe `feature_branch_2`, trebuie data comanda de rebase pe branch-ul `feature_branch_2`

Pentru următoarele exerciţii, veţi lucra pe diferite branch-uri din acest repository, cu un program scris de un intern imaginar, de la o firmă imaginară. Programul trebuie să citească un număr de la tastatură şi să întoarcă valoarea citită adunată cu 5. Pentru fiecare exerciţiu, veţi crea un pull request către branch-ul **main** din fork-ul vostru.

- Folosind **git commit --amend**, modificaţi mesajul de commit al ultimului commit de pe branch-ul **amend_branch**. Nu contează în ce modificaţi mesajul.
- Pe parcursul dezvoltării, internul a introdus un bug: numărul citit nu se adună cu 5, ci cu 6. Folosind `git bisect`, găsiţi commit-ul care a introdus bug-ul. (Nu contează dacă e evident unde apare, folosiţi **bisect**). Adăugaţi un commit care rezolvă problema, pe branch-ul **bisect_branch**.

Considerati commit-ul good ca fiind primul commit (1e598f4257cd3b8711b630693780914fb76754e6)

- Folosind **git rebase --interactive**, faceţi **squash** commit-urilor de pe branch-ul **rebase_interactive_branch** astfel incat sa fie frumos commit history-ul (ideal doar commit-uri care aduc functionalitati in plus, nu repara chestii).
- Folosind **git reset**, eliminaţi toate commit-urile de pe branch-ul **reset_soft_branch** şi creaţi un singur commit cu întreaga implementare.
- Manager-ul intern-ului tocmai a zis ce task avea de fapt de făcut: să genereze un numar random în 5 moduri diferite. Folosind **git reset --hard** Eliminaţi toate modificările de pe branch-ul **reset_hard_branch** şi implementaţi ce i s-a cerut de fapt. (E o mica provocare să gasiti a 5-a metoda - nu citire din **/dev/random** sau **rdseed**).

Feedback

Dacă ceva nu a mers sau aţi mai fi dorit să fie şi alte chestii în acest laborator, adăugati un issue.

Resurse utile

- https://gitimmersion.com/ [https://gitimmersion.com/]
- https://git-scm.com/docs/ [https://git-scm.com/docs/]

Referințe

- https://www.atlassian.com/git/tutorials/what-is-version-control [https://www.atlassian.com/git/tutorials/what-is-version-control]
- https://teaching.alexcoman.com/resurse/tutorial/git/sisteme-pentru-controlul-versiunii/ [https://teaching.alexcoman.com/resurse/tutorial/git/sisteme-pentru-controlul-versiunii/]
- https://www.cs.odu.edu/~zeil/cs350/latest/Public/versionControl/index.html [https://www.cs.odu.edu/~zeil/cs350/latest/Public/versionControl/index.html]

tsc/laboratoare/laborator-04.txt · Last modified: 2024/04/03 14:33 by giorgiana.vlasceanu